

Developer manual

(Onvif Rtsp Server)

HAPPYTIME SOFTWARE CO., LIMITED

Declaration

All rights reserved. No part of this publication may be excerpted, reproduced, translated, annotated or edited, in any form or by any means, without the prior written permission of the copyright owner.

Since the product version upgrade or other reasons, this manual will subsequently be updated. Unless otherwise agreed, this manual only as a guide, this manual all statements, information, recommendations do not constitute any express or implied warranties.

www.happytimesoft.com

Table of Contents

Chapter 1 Build	5
1.1 Dependent libraries.....	5
1.2 Windows Platform.....	5
1.3 Linux Platform.....	5
1.4 MAC Platform.....	5
1.5 Build APK for android.....	6
1.6 Embedded Platform.....	6
1.7 Compile PROFILE T compatible version.....	7
1.8 Compile PROFILE S compatible version.....	7
1.9 Compile PROFILE G compatible version.....	8
1.10 Compile ONVIF Server without RTSP Server.....	8
1.11 Compile RTSP Server without ONVIF Server.....	9
Chapter 2 Compilation options.....	10
Chapter 3 Build ALSA library on Linux platform.....	13
Chapter 4 Install XCB library on Linux platform.....	14
Chapter 5 ONVIF API Interface.....	15
Chapter 6 ONVIF Development	16
6.1 Initialize the system BUFFER.....	16
6.2 Initialize device capabilities.....	16
6.3 Initialize audio and video encoding parameters.....	17
6.4 Other initialization.....	17
6.5 Device UUID.....	17
6.6 Configuration file.....	18
6.7 Get the rtsp stream address of the profile.....	18
6.8 Get profile snapshot.....	18
6.9 Maximum number of profiles.....	19
6.10 Timer task.....	19
6.11 Onvif Event.....	20
6.12 Handling new ONVIF interfaces.....	20
Chapter 7 Embedded developers	21
7.1 Compile.....	21
7.2 Feed custom data.....	21
7.3 Support multiple streams.....	22
7.4 Cutting size.....	23
Chapter 8 RTSP Development	24
8.1 Initialize the system BUFFER.....	24
8.2 Audio SDP description.....	24
8.3 Video SDP description.....	25

8.4 RTP packet.....	26
Chapter 9 RTSP over HTTP	28
Chapter 10 RTSP over Websocket.....	30
Chapter 11 DLL files.....	31
Chapter 12 Android development.....	32
12.1 Set the Android compilation environment.....	32
12.2 Configure project.....	32
12.3 Android extra.....	33
12.4 Android manifest.....	34
12.5 Build Release APK.....	34

Chapter 1 Build

1.1 Dependent libraries

The Onvif Rtsp server depend on the following libraries:

ffmpeg 4.0 (When the MEDIA_FILE or MEDIA_DEVICE compilation option is enabled, it requires linking to the ffmpeg library, on embedded platform, if you do live streaming, you don't need ffmpeg library)

ALSA 1.1.6 (Stream from audio device on Linux platform, when the MEDIA_DEVICE compilation option is enabled, it requires linking to the ALSA library)

XCB 1.9 (Stream from live screen or application windows on Linux platform, when the MEDIA_DEVICE compilation option is enabled, it requires linking to the XCB library)

SRT 1.5.4 (Proxy SRT stream, when the SRT_PROXY compilation option is enabled, it requires linking to the SRT library)

Openssl 1.1.1g (When the HTTPS or RTSPS or RTMP_PROXY or SRT_PROXY compilation option is enabled, it requires linking to the Openssl library)

ZLIB 1.2.11

SRTP 2.6.0 (Encrypt data transmission using security RTP, when the RTSP_SRTP or SRTP compilation option is enabled, it requires linking to the SRTP library)

1.2 Windows Platform

Use VS2019 open OnvifRtspServer.sln to build.

1.3 Linux Platform

Note: [ALSA](#) and [XCB](#) libraries need to be installed before compilation.

```
make clean
```

```
make
```

1.4 MAC Platform

```
make -f mac.mk clean
```

```
make -f mac.mk
```

Note: Please use the brew command to install third-party libraries:

```
brew install ffmpeg
```

```
brew install srt
```

```
brew install srtp2
```

```
brew install openssl
```

Note: Support ffmpeg 4.x, 5.x and 6.x version.

1.5 Build APK for android

Use QtCreator open android.pro to build, please reference [Android development](#).

1.6 Embedded Platform

Modify embed.mk to specify cross-compiler, run the following commands to compile, please reference [Embedded developers](#):

```
make -f embed.mk clean
```

```
make -f embed.mk
```

Compile PROFILE G version

```
make -f embed-profileg.mk clean
```

```
make -f embed-profileg.mk
```

Compile PROFILE M version

```
make -f embed-profilem.mk clean
```

```
make -f embed-profilem.mk
```

Compile PROFILE S version

```
make -f embed-profiles.mk clean
```

```
make -f embed-profiles.mk
```

Compile PROFILE T version

```
make -f embed-profilet.mk clean
```

```
make -f embed-profilet.mk
```

Please modify the following options to specify the cross compiler:

CCOMPILER -- Specify the C file compiler.

CPPCOMPILER -- Specify the C++ file compiler.

LINK -- Specify the object file linker.

COMPILEOPTION -- Add compile options.

LINKOPTION -- Add link options.

INCLUDEDIR -- Add header file search path.

LIBDIRS -- Add library file search path.

SHAREDLIB -- Add library files that need to be linked.

Note: If you enable HTTPS, you need to cross-compile openssl library, then modify LIBDIRS to point to the openssl library path that you cross compile.

Note: Cross compile openssl library on embedded platform, please turn off assembly code with no-asm option.

If you don't need to enable HTTPS function, you can comment out the HTTPS option.

1.7 Compile PROFILE T compatible version

Windows Platform:

Open the OnvifRtspServer.sln, select OnvifRtspServerProfileT project to build.

Linux platform:

```
make -f profilet.mk clean
```

```
make -f profilet.mk
```

MAC platform:

```
make -f mac-profilet.mk clean
```

```
make -f mac-profilet.mk
```

1.8 Compile PROFILE S compatible version

Windows Platform:

Open the OnvifRtspServer.sln, select OnvifRtspServerProfileS project to build.

Linux platform:

```
make -f profiles.mk clean
```

```
make -f profiles.mk
```

MAC platform:

```
make -f mac-profiles.mk clean
```

```
make -f mac-profiles.mk
```

1.9 Compile PROFILE G compatible version

Windows Platform:

Open the OnvifRtspServer.sln, select OnvifRtspServerProfileG project to build.

Linux platform:

```
make -f profileg.mk clean
```

```
make -f profileg.mk
```

MAC platform:

```
make -f mac-profileg.mk clean
```

```
make -f mac-profileg.mk
```

1.10 Compile ONVIF Server without RTSP Server

Windows Platform:

Open the OnvifRtspServer.sln, select OnvifServer project to build.

Linux platform:

```
make -f onvifserver.mk clean
```

```
make -f onvifserver.mk
```

MAC platform:

```
make -f onvifserver-mac.mk clean
```

```
make -f onvifserver-mac.mk
```

Android platform:

```
make -f onvifserver-android.mk clean
```

```
make -f onvifserver-android.mk
```

Note: Need to install android build environment.

Note: Please modify NDK in android.mk to your NDK path.

IOS platform:

```
make -f onvifserver-ios.mk clean
```

```
make -f onvifserver-ios.mk
```

Note: Need to install iOS build environment.

1.11 Compile RTSP Server without ONVIF Server

Windows Platform:

Open the OnvifRtspServer.sln, select RtspServer project to build.

Linux platform:

```
make -f rtspserver.mk clean
```

```
make -f rtspserver.mk
```

MAC platform:

```
make -f rtspserver-mac.mk clean
```

```
make -f rtspserver-mac.mk
```

Embedded platform:

```
make -f rtspserver-embed.mk clean
```

```
make -f rtspserver-embed.mk
```

Chapter 2 Compilation options

The VS project and Makefile support the following compile options:

EPOLL: Enable EPOLL mode for socket multiplexing on Linux.

HTTPS: Support HTTPS feature, it requires linking to the openssl library.

HTTPD: Implementing web service.

PROFILE_G_SUPPORT: Implement ONVIF profile G related interfaces, include ONVIF recording, search, replay services, it is a mandatory option to support ONVIF PROFILE G.

PROFILE_C_SUPPORT: Implement ONVIF profile C related interfaces, include ONVIF accesscontrol, doorcontrol service, it is a mandatory option to support ONVIF PROFILE C.

CREDENTIAL_SUPPORT: Implement ONVIF credential service interfaces.

ACCESS_RULES: Implement ONVIF accessrules service interfaces.

SCHEDULE_SUPPORT: Implement ONVIF schedule service interfaces.

AUDIO_SUPPORT: Support audio input and output.

DEVICEIO_SUPPORT: Implement ONVIF deviceio service interfaces.

IMAGE_SUPPORT: Implement ONVIF imaging service interfaces.

MEDIA_SUPPORT: Implement ONVIF media 1 service interfaces, it is a mandatory option to support ONVIF PROFILE S.

MEDIA2_SUPPORT: Implement ONVIF media 2 service interfaces, it is a mandatory option to support ONVIF PROFILE T.

MPEG4_SUPPORT: Support MPEG4 video encoding configuration, MPEG4 video encoding is optional for ONVIF PROFILE S and ONVIF PROFILE T.

PTZ_SUPPORT: Implement ONVIF PTZ service interfaces.

VIDEO_ANALYTICS: Implement ONVIF analytics service interfaces, it is a mandatory option to support ONVIF PROFILE M.

THERMAL_SUPPORT: Implement ONVIF thermal service interfaces.

RECEIVER_SUPPORT: Implement ONVIF receiver service interfaces.

IPFILTER_SUPPORT: Implement IP filter interfaces.

STORAGE_SUPPORT: Implement storage configuration and management interfaces.

PROVISIONING_SUPPORT: Implement ONVIF provisioning service interfaces.

PROFILE_Q_SUPPORT: Implement ONVIF Profile Q interfaces, it is a mandatory

option to support ONVIF PROFILE Q.

GEOLOCATION_SUPPORT: Implement geolocation configuration and management interfaces.

DOT11_SUPPORT: Implement dot11 configuration and management interfaces.

MEDIA_FILE: Support streaming from local media files, it requires linking to the ffmpeg library.

MEDIA_DEVICE: Support streaming from devices (camera, microphone, live screen, application windows), it requires linking to the ffmpeg library.

MEDIA_PUSHER: Support stream data publish/input by RTSP/TCP/UDP protocol.

MEDIA_PROXY: Support streaming from other media servers (rtsp server, http mjpeg server).

RTMP_PROXY: Support streaming from RTMP servers (Need to enable MEDIA_PROXY), it requires linking to the openssl library.

SRT_PROXY: Support streaming from SRT servers (Need to enable MEDIA_PROXY), it requires linking to the openssl library.

MEDIA_LIVE: Support streaming from encoded media data (Living stream mode, for embedded platform).

RTSP_OVER_HTTP: Support RTSP over HTTP function.

HTTPS: Support RTSP over HTTPS function, it requires linking to the openssl library.

RTSP_RTCP: Support RTCP function, send RTCP packets.

RTSP_METADATA: Support RTSP metadata stream.

RTSP_BACKCHANNEL: Support audio back channel.

RTSP_REPLAY: Support RTSP replay function (According to the ONVIF media replay specification)

RTSP_S RTP: Support to encrypt data transmission using security RTP, it requires linking to the SRTP library.

RTSP_OVER_WEBSOCKET: Support RTSP over WebSocket (Need to enable RTSP_OVER_HTTP).

OVER_HTTP: Support rtsp over http (For rtsp client of media proxy)

OVER_WEBSOCKET: Support rtsp over WebSocket (For rtsp client of media proxy)

METADATA: Support rtsp metadata stream (For rtsp client of media proxy)

HTTP_NOTIFY: Support HTTP event notification callback.

SRTP: Support to encrypt data transmission using security RTP (For rtsp client of media proxy), it requires linking to the srtplib library.

RTSPS: Support secure RTSP (RTSP over TLS/SSL), it requires linking to the openssl library.

NO_ONVIF_SERVER: Do not compile onvif server code.

NO_RTSP_SERVER: Do not compile rtsp server code.

Chapter 3 Build ALSA library on Linux platform

If you enable MEDIA_DEVICE compiler option on Linux platform, you need to build ALSA library, please run the following commands to build:

```
cd third  
tar jxf alsa-lib-1.1.6.tar.bz2  
cd alsa-lib-1.1.6  
./configure --enable-shared=yes  
make install
```

On 64bit Linux platform:

```
ln -s /usr/lib64/alsa-lib /usr/lib/alsa-lib
```

Chapter 4 Install XCB library on Linux platform

If you enable MEDIA_DEVICE compiler option on Linux platform, you need to install XCB library.

On CentOS system, please run the following commands to install:

```
yum install libxcb
```

```
yum install libxcb-devel
```

On ubuntu system, please run the following commands to install:

```
sudo apt install xcb
```

```
sudo apt install libxcb-util0-dev
```

Chapter 5 ONVIF API Interface

Onvif client request handler showed in the following table:

Service	In file
Device Management	onvif_device.c
Media Service	onvif_media.c
Media Service 2	Onvif_media2.c
Image Service	onvif_image.c
PTZ Service	onvif_ptz.c
Event Service	onvif_event.c
Recording Service Search Service Replay Service	onvif_recording.c
Analytics Service	onvif_analytics.c
Door control service Access control service	onvif_doorcontrol.c
Access Rules service	onvif_accessrules.c
Credential service	onvif_credential.c
Device IO service	onvif_deviceio.c
Thermal service	onvif_thermal.c
Schedule service	onvif_schedule.c
Provisioning service	Onvif_provisioning.c
Receiver service	onvif_receiver.c
Security service	onvif_security.c

Each client request has been done the basic processing, such as parameters checking, parameters saving, if the request need to call the system interface, such as SystemRoot etc., you need add code to complete the request.

Note: Each ONVIF standard request has onvif_ prefix.

Chapter 6 ONVIF Development

6.1 Initialize the system BUFFER

When the onvif server starts, it needs to allocate the system BUFFER and the HTTP message parser BUFFER. The system BUFFER is mainly used to receive network data, and the HTTP message parser BUFFER is used to parse HTTP messages. The size of each system BUFFER is 2048 bytes, and the size of each HTTP message parser BUFFER is about 512 bytes. These BUFFERS are allocated when the system starts, network data reception and message parse do not require dynamic memory allocation.

The onvif server allocates 32 system BUFFER and 16 HTTP message parser BUFFER by default. Users can adjust these parameters. For details, please refer to the main function:

```
int main(int argc, char * argv[])
{
    ...
    sys_buf_init(32);
    http_msg_buf_init(16);
    ...
}
```

6.2 Initialize device capabilities

ONVIF version 1.0 uses the GetCapabilities interface to obtain the device capability set, and GetCapabilities is the device service interface.

ONVIF versions 1.0 above use the GetService or GetServiceCapabilities interface to obtain the device capability set. Each ONVIF service must support GetServiceCapabilities interface.

The onvif server initializes the capability set of the device in the **onvif_init_capabilities** function, and the code of this function can be modified according to the actual situation of the device.

There is a support field in each service capability set structure. If support field is set to 0, it means that the service is not supported, such as:

```
g_onvif_cfg.Capabilities.media2.support = 0;
```

Indicates that the onvif media2 service is not supported.

Each ONVIF service needs to provide a service address. The service address of ONVIF device service must be fixed /onvif/device_service, and other ONVIF service addresses can be defined by themselves.

Some VMS systems use the number of VideoSources in the deviceio service capability to treat the device as a single channel (VideoSources=1) or multi-channel device (VideoSources>1).

If the device supports audio output, there must be at least one AudioOutput in the deviceio service capability. If the device supports the RTSP audio back-channel function, there must be at least one AudioSources in the deviceio service capability.

6.3 Initialize audio and video encoding parameters

The system supports the following video encodings by default: H264, H265, JPEG, MP4, and the following audio encodings: G711, G726, AAC.

You can comment out the unsupported audio and video encoding according to your situation.

Note: ONVIF PROFILE S must support H264, JPEG video encoding, ONVIF PROFILE T must support H264, H265, JPEG video encoding.

Please refer to the specific code:

```
onvif_init_VideoEncoderConfigurationOptions
```

```
onvif_init_AudioEncoderConfigurationOptions
```

6.4 Other initialization

For other initialization, please refer to the **onvif_init_cfg** function, please modify it according to your actual situation.

6.5 Device UUID

The device UUID is the unique identification of the device. The VMS system should use the UUID of the device as the unique identification of the device. Even if the device IP changes, a device can be uniquely identified through the device UUID.

The client can obtain the UUID of the device through the probe response message of ONVIF discovery or call the GetEndpointReference interface to obtain the UUID of the device.

6.6 Configuration file

When run onvif server for the first time, use the default configuration file `onvif.cfg`, which sets 2 profiles.

When stop onvif server, it writes the runtime configurations into the `Onvifrun.cfg` file, and the configurations in the `onvifrun.cfg` file will be load at the next time it runs.

If you modify the default configuration file `onvif.cfg`, you should stop the onvif server first, then delete the runtime configuration `onvifrun.cfg`, and run onvif server again to make the default configuration effective.

If you are familiar with the runtime configuration file `onvifrun.cfg`, you can also modify it.

6.7 Get the rtsp stream address of the profile

The ONVIF client tool calls the `GetStreamUri` interface to obtain the RTSP stream address of the profile. If the rtsp stream address of the profile is not configured in the configuration file, Onvif server returns a test address `rtsp://ip/test.mp4` by default. If the rtsp stream address of the profile is configured in the configuration file, it will return the configured rtsp stream address.

The test rtsp stream address will also have audio and video parameter information, such as video encoding, video width, video height, audio encoding, audio sampling rate, number of channels, transmission protocol, etc. If the transmission protocol is HTTP (RTSP over HTTP), the returned test address is `http://ip/test.mp4`.

Users should modify the returned test address according to their situation. For details, please refer to `onvif_trt_GetStreamUri` (`GetStreamUri` interface of onvif media 1 service) and `onvif_tr2_GetStreamUri` (`GetStreamUri` interface of onvif media 2 service) functions.

6.8 Get profile snapshot

If the media service capability set of the device supports get snapshot, the onvif client tool calls the `GetSnapshot` interface to obtain the snapshot url, and then requests the snapshot through http get request.

The onvif server reads the `snapshot.jpg` file content as a snapshot and returns it to the client by default. The user should modify the returned content. For details, please refer to `soap_GetSnapshot`, `onvif_trt_GetSnapshot` functions.

6.9 Maximum number of profiles

When onvif client tool calls GetProfiles, it needs to return all profile parameters, so the response of GetProfiles request will be large. The default maximum message buffer size of onvif server is 40KB, which can support about 10 profiles.

If you need to support more profiles, you need to modify the maximum message buffer size, please refer to the soap_build_send_rly function:

```
ONVIF_RET soap_build_send_rly(...)
{
    int offset = 0;
    int ret = -1, mlen = 1024*40, xlen;

    // todo : The default maximum message buffer size is 40KB,
    // which supports a maximum of 10 profiles.
    // If you need to support more profiles, please modify mlen to a larger
    value

    char * p_xml = (char *)malloc(mlen);
    if (NULL == p_xml)
    {
        return (ONVIF_RET)-1;
    }
    ...
}
```

6.10 Timer task

Users can add or delete timer processing through the timer_add and timer_del interfaces. These processing are scheduled through the main task thread. All timer-related tasks are processed in the onvif_timer function, including timer processing added by the user, onvif event subscription renew processing, onvif PullMessage request timeout processing, onvif SCHEDULE service processing, onvif search service timeout processing, etc. For details, please refer to the implementation of onvif_timer function.

6.11 Onvif Event

If you need to send an onvif event, please construct an onvif event message first.

Please reference the following functions:

onvif_init_NotificationMessage, onvif_init_NotificationMessage1,
onvif_init_NotificationMessage2, onvif_init_NotificationMessage3,
onvif_init_NotificationMessage4.

Each standard onvif event has its own message property data, you need to refer to the official onvif standard document or refer to the build_tev_GetEventProperties_rly_xml function.

After construct the onvif event, call the **onvif_put_NotificationMessage** interface to send the event.

For the subscription mode, the event will be sent to the subscribing client immediately; for the polling mode, the event will be placed in the event queue, waiting for the next time the client calls PullMessages request to be returned to the client.

The default size of the polling mode event queue is 20. When the queue is full, the earliest incoming event will be deleted.

6.12 Handling new ONVIF interfaces

1. In onvif_<custom_service>

1.1 Make request struct with needed parameters.

1.2 Add request functions and handler code.

2. In soap.c:

2.1 soap_process_<custom_service>_request define and add new function for soap request.

2.2 Call parsing function for request.

2.3 Call handler code if parsing success (onvif_<custom_service>)

3. In soap_parser.c

3.1 Add parsing code for function that gets parameter values from the xml

3.2 Check match with WSDL data type.

4. In onvif_pkt.c

4.1 Build xml response:

build_<custom_service_namespace>_<function_name>_rly_xml

Chapter 7 Embedded developers

7.1 Compile

On embedded platform, please use `embed.mk` to compile (You need to specify your cross compiler before compiling):

```
make -f embed.mk clean  
make -f embed.mk
```

Please modify the following options in `embed.mk` file to specify your cross compiler:

```
CCOMPILER -- Specify the C file compiler.  
CPPCOMPILER -- Specify the C++ file compiler.  
LINK -- Specify the object file linker.  
COMPILEOPTION -- Add compile options.  
LINKOPTION -- Add link options.  
INCLUDEDIR -- Add header file search path.  
LIBDIRS -- Add library file search path.  
SHAREDLIB -- Add library files that need to be linked.
```

7.2 Feed custom data

In `rtsp_parse_live_url` function, parse the URL suffix, set the `p_rua->media_info` struct.

```
p_rua->media_info.v_index = 0;
```

Set video encoder index, the index will be passed into `CLiveVideo::m_nStreamIndex`.

```
p_rua->media_info.a_index = 0;
```

Set audio encoder index, the index will be passed into `CLiveAudio::m_nStreamIndex`.

The frame rate and resolution of the video, if there is audio, the sampling rate and number of channels of the audio can be obtained from the hardware encoder.

In `live_video.cpp` file, implement the following functions:

```
initCapture()  
stopCapture()  
captureThread()
```

If has audio, in **live_audio.cpp** file, implement the following functions:

initCapture()

stopCapture()

captureThread()

Note: Implement captureThread function, call **procData** function after getting audio and video encoded data

CLiveVideo and CLiveAudio are stub class, the purpose is to obtain encoded data from the hardware encoder and send it to the video and audio transmission queue.

When the rtsp client connects for the first time, call the initCapture interface, where the hardware encoder can be initialized.

When the last rtsp client disconnects, call stopCapture(), here you can stop the hardware encoder.

captureThread, get the encoded data and then call procData to send the data to the video and audio queue.

If you have your own data capture thread, you can also call **media_live_put_video** and **media_live_put_audio** to send data to the video and audio queue.

7.3 Support multiple streams

If you want to support the following 2 stream addresses:

rtsp://ip/live1 (HD)

rtsp://ip/live2 (4K)

First, in the **rtsp_parse_live_url** function, set the media information and the index, such as:

```
if (strncasecmp(p_url, "live1", strlen("live1")) == 0)
{
    p_rua->media_info.is_live = 1;

    p_rua->media_info.has_video = 1;
    p_rua->media_info.v_index = 0;
    p_rua->media_info.v_info.framerate = 25;
```

```

        p_rua->media_info.v_info.width = 1280;
        p_rua->media_info.v_info.height = 720;
        p_rua->media_info.v_info.codec = VIDEO_CODEC_H264;
    }
    else if (strncasecmp(p_url, "live2", strlen("live2")) == 0)
    {
        p_rua->media_info.is_live = 1;

        p_rua->media_info.has_video = 1;
        p_rua->media_info.v_index = 1;
        p_rua->media_info.v_info.framerate = 25;
        p_rua->media_info.v_info.width = 1920;
        p_rua->media_info.v_info.height = 1080;
        p_rua->media_info.v_info.codec = VIDEO_CODEC_H264;
    }

```

If there is audio, specify the audio encoder index through `p_rua->media_info.a_index`.

If there is only one audio encoding stream, multiple streams can multiplex the same audio.

In the `getStreamNums` function in `live_video.cpp`, modify the support stream numbers, such as:

```

CLiveVideo::getStreamNums()
{
    return 2;
}

```

In the `initCapture` and `captureThread` function, use the Member variables `m_nStreamIndex` to initialize and capture the streams.

7.4 Cutting size

In `rtsp_rsua.h`, adjust the `MAX_NUM_RUA` to fit your requirements. The `MAX_NUM_RUA` means the maximum number of clients supported.

Chapter 8 RTSP Development

8.1 Initialize the system BUFFER

When rtsp server starts, it needs to allocate system BUFFER and RTSP message parsing BUFFER. System BUFFER is used to receive network data. RTSP message parsing BUFFER is used to parse RTSP messages. The size of each system BUFFER is 2048 bytes, the size of each RTSP message parses BUFFER is about 160 bytes. These BUFFERS are allocated when the system starts, network data reception and message parse do not require dynamic memory allocation.

The rtsp server allocates 2 times MAX_NUM_RUA system BUFFER and RTSP message parsing BUFFER by default. Users can adjust these parameters. For details, please refer to the `rtsp_start1` function:

```
sys_buf_init(bufs);  
rtsp_parse_buf_init(bufs);
```

If the MEDIA_PROXY or RTSP_OVER_HTTP function is enabled, the system also needs to allocate HTTP message parsing BUFFER. HTTP message parsing BUFFER is used to parse HTTP messages. The size of each HTTP message parsing BUFFER is about 512 bytes.

```
http_msg_buf_init(bufs);
```

When the system exits, call the following functions to release the system BUFFER and RTSP, HTTP message parsing BUFFER.

```
rtsp_parse_buf_deinit();  
sys_buf_deinit();  
http_msg_buf_deinit();
```

Please refer to the **rtsp_stop** function.

8.2 Audio SDP description

When the audio encoding is G711, G722, G726, OPUS, the SDP description in the response message of the DESCRIBE request does not require a format parameter, examples are as follows:

```
a=rtpmap:0 PCMU/44100/2 // G711 ulaw, 44100 sampling rate, 2 channels
```


When the audio encoding is AAC, the format parameters are required. Examples are as follows:

```
a=rtpmap:97 MPEG4-GENERIC/44100/2 // AAC, 44100 sampling rate, 2 channels
a=fmtp:97
streamtype=5;profile-level-id=1;mode=AAC-hbr;size-length=13;index-length=3;indexd
elt-length=3;config=121056E500
```

The getAACAuxSDPLine function of the CAudioEncoder class is used to construct the AAC format parameters:

```
a=fmtp:97
streamtype=5;profile-level-id=1;mode=AAC-hbr;size-length=13;index-length=3;indexd
elt-length=3;config=121056E500
```

Where m_pCodecCtx->extradata points to AAC specification data, and m_pCodecCtx->extradata_size represents the AAC specification data size. These data are generated by the encoder after the encoder is initialized.

For embedded developers, you need to implement the getAuxSDPLine function of the CLiveAudio class to construct the AAC format parameters. The AAC specification parameters can be obtained from the hardware encoder, or the AAC format parameters can be constructed according to the sampling rate and the number of channels. For details, please refer to CLiveAudio Class getAuxSDPLine function.

8.3 Video SDP description

When the video encoding is JPEG, the SDP description in the response message of the DESCRIBE request does not require a format parameter, examples are as follows:

```
a=rtpmap:26 JPEG/90000
```

When the video encoding is MP4, the SDP description in the response message of the DESCRIBE request require a format parameter, examples are as follows:

```
a=rtpmap:96 MP4V-ES/90000
a=fmtp:96
profile-level-id=1;config=000001B001000001B58913000001000000012000C48D8800CD280
45A1463000001B24C61766335382E31382E313030
```

The getMP4AuxSDPLine function of the CVideoEncoder class is used to construct MP4 format parameters.

When the video encoding is H264, the SDP description in the response message of the DESCRIBE request require a format parameter, examples are as follows:

```
a=rtpmap:96 H264/90000  
a=fmtp:96  
packetization-mode=1;profile-level-id=00001F;sprop-parameter-sets=Z0KAH5ZSAKALd  
JQEBAUAAAMAAQAAAwAyhA==,aMuNSA==
```

The getH264AuxSDPLine function of the CVideoEncoder class is used to construct H264 format parameters.

m_pCodecCtx->extradata points to the SPS and PPS parameters of H264. For embedded developers, the SPS and PPS parameters can be obtained from the hardware encoder.

When the video encoding is H265, the SDP description in the response message of the DESCRIBE request require a format parameter, examples are as follows:

```
a=rtpmap:96 H265/90000  
a=fmtp:96  
profile-space=0;profile-id=1;tier-flag=0;level-id=93;interop-constraints=900000  
000000;sprop-vps=QAEMAf//AWAAAAMakAAAAwAAAwBdugJA;sprop-sps=QgEBAWAAAAMakAAAAwA  
AAwBdoAKAgCOWW6uTC4BAAAADAEAAAACZC;sprop-pps=RAHBcYMS
```

The getH265AuxSDPLine function of the CVideoEncoder class is used to construct H265 format parameters.

m_pCodecCtx->extradata points to the VPS, SPS, and PPS parameters of H265. For embedded developers, the VPS, SPS, and PPS parameters can be obtained from the hardware encoder.

8.4 RTP packet

For AAC encoded data, the ADTS header is not needed during RTP transmission. If the AAC encoded data obtained from the audio encoder has the ADTS header, the ADTS header needs to be removed.

For H264 and H265 encoded data, if the SDP description of the response message of the DESCRIBE request has format parameters, it is not necessary to send the VPS, SPS, PPS parameters. If the SDP description of the response message of the DESCRIBE request does not have the format parameters, you need to send the VPS, SPS, PPS parameters, otherwise RTSP client cannot decode the data after receiving it.

Chapter 9 RTSP over HTTP

The key of RTSP over HTTP is to allow RTSP packets to communicate via HTTP port.

We know that the standard port of RTSP is 554, but due to various security policy configurations such as firewalls, there may be restrictions when the client accesses port 554, which prevents the normal transmission of RTSP packets.

But the HTTP port (port 80) is generally open, so there is the idea of letting RTSP packets pass through port 80, namely RTSP over HTTP

The details of RTSP over HTTP are as follows:

First, the client opens two sockets connect to the rtsp server HTTP ports. We call these two sockets "data socket" and "command socket".

Step 1. The client sends an HTTP GET command through the "data socket" to request an RTSP connection.

Step 2. The server responds to the HTTP GET command through the "data socket" and responds with success/failure.

Step 3. The client creates a "command socket" and sends an HTTP POST command through the "command socket" to establish an RTSP session.

At this point, the auxiliary function of HTTP is completed, and the server does not return the client's HTTP POST command. Next is the standard process of RTSP on the HTTP port, but it needs to be completed through two sockets. The "command socket" is only responsible for sending, and the "data socket" is only responsible for receiving.

Step 4. The client sends RTSP commands (BASE64 encoding) through the "command socket".

Step 5. The server responds to the RTSP command (in plain text) through the "data socket".

Step 6. Repeat Step4-Step5 until the client sends the RTSP PLAY command and the server responds to the RTSP PLAY command.

Step 7. The server transmits audio and video data to the client through the "data socket".

After the data exchange is complete...

Step 8. The client sends the RTSP TEARDOWN command (BASE64 encoding and) through the "command socket".

Step 9. The server responds to the RTSP TEARDOWN command (in plain text) through

the "data socket".

Step 10. Close the two sockets.

Chapter 10 RTSP over WebSocket

First establish an HTTP connection, and then upgrade to the WebSocket protocol, RTSP over WebSocket protocol upgrade process:

Client-->Server:

GET /websocket HTTP/1.1 Host: 192.168.3.27

Upgrade: websocket Connection: Upgrade

Sec-WebSocket-Key: KS0+h0Fslq5SkEnx8bvp6w==

Origin: http://192.168.3.27

Sec-WebSocket-Protocol: rtsp.onvif.org Sec-WebSocket-Version: 13

Server-->Client:

HTTP/1.1 101 Switching Protocols Upgrade: websocket

Connection: Upgrade

Sec-WebSocket-Accept: G/cEt4HtsYEnPOMnSVkKRk459gM=

Sec-WebSocket-Protocol: rtsp.onvif.org

Sec-WebSocket-Version: 13

After the protocol upgrade is successful, perform normal rtsp protocol exchange, send and receive data through WebSocket connection.

Chapter 11 DLL files

On windows platform, after the successful compilation of the project, if running or debugging, there is no dynamic library file, please download our demo version from the following link:

x86 version:

<https://happytimesoft.com/downloads/happytime-onvif-rtsp-server.zip>

x64 version:

<https://happytimesoft.com/downloads/happytime-onvif-rtsp-server-x64.zip>

Then copy all DLL files from the installed path to your project bin path.

Chapter 12 Android development

12.1 Set the Android compilation environment

1. Install QT 6.5.1 for Android:

https://download.qt.io/official_releases/online_installers/qt-unified-windows-x64-online.exe

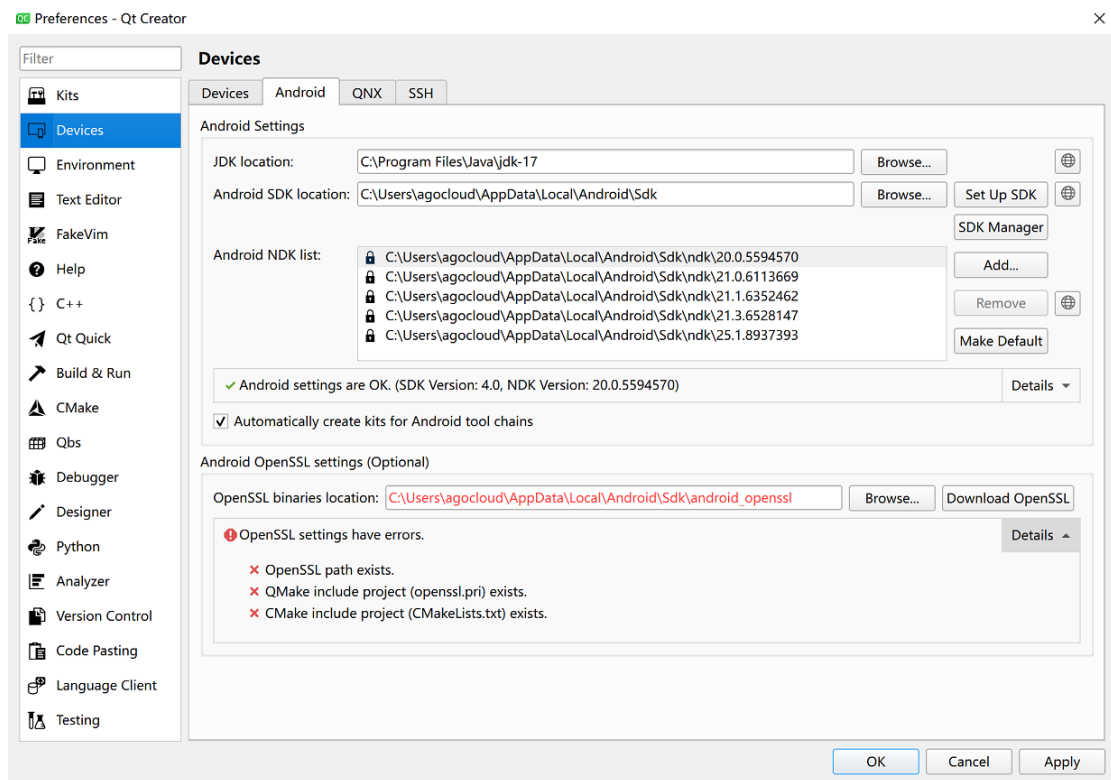
2. Install android studio, configure Android SDK, NDK

The Android app has the following requirements:

JDK 17

Android platform SDK 33

3. Start QtCreator, Open "Edit->Preferences" menu item, select "devices", set the "JDK Location", "Android SDK Location" and "Android NDK Location", as the following:



4. The Android compile environment setup is complete.

12.2 Configure project

Open *.pro for the first time, and select android for the project configuration instead of desktop:

Configure Project

The following kits can be used for project **OnvifClientForAndroid**:

☒ Select all kits	Type to filter kits by name...
☒ Android Qt 6.5.1 Clang arm64-v8a	Details ▾
☒ Android Qt 6.5.1 Clang armeabi-v7a	Details ▾
☐ Android Qt 6.5.1 Clang x86	Manage... Details ▾
☐ Android Qt 6.5.1 Clang x86_64	Manage... Details ▾
☐ Desktop Qt 6.5.1 MSVC2019 64bit	Manage... Details ▾
☐ Desktop Qt 6.5.1 MinGW 64-bit	Manage... Details ▾
☐ Replacement for "Android Qt 5.15.0 Clang Multi-Abi"	Manage... Details ▾
Import Build From...	Details ▾

[Configure Project](#)

Delete *.pro.user in the directory, and then open *.pro, the project configuration window will pop up again.

12.3 Android extra

The android extra code in
android/src/org/happytimesoft/util/HtUtil.java

enableMulticast:

Turn on the multicast reception, android system order to save power, the default does not receive the multicast packets.

disableMulticast:

Turn off the multicast reception.

openFile:

Call the third-party application to open the file.

disableLockScreen:

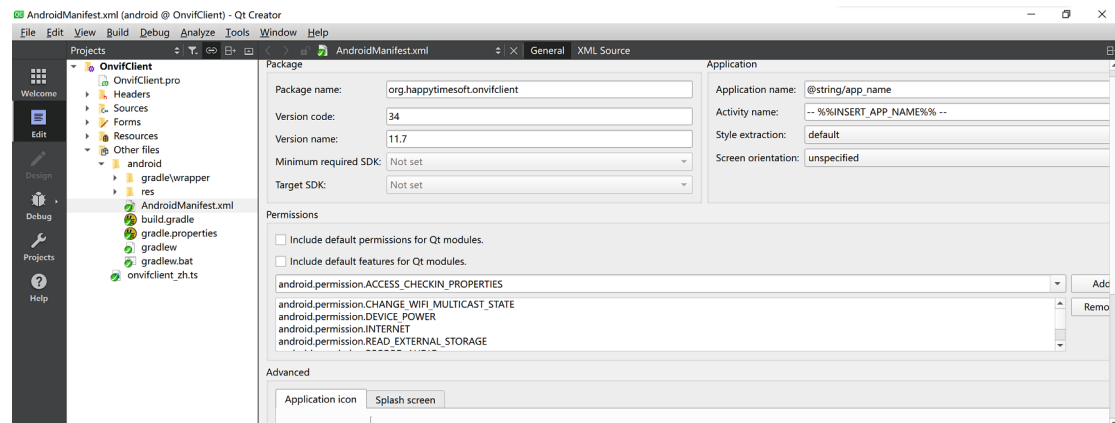
Turn off the automatic lock screen function.

enableLockScreen:

Turn on the automatic lock screen function.

12.4 Android manifest

Open AndroidManifest.xml in QtCreator, as the following:



You can do the following modifications:

Packet name

Version

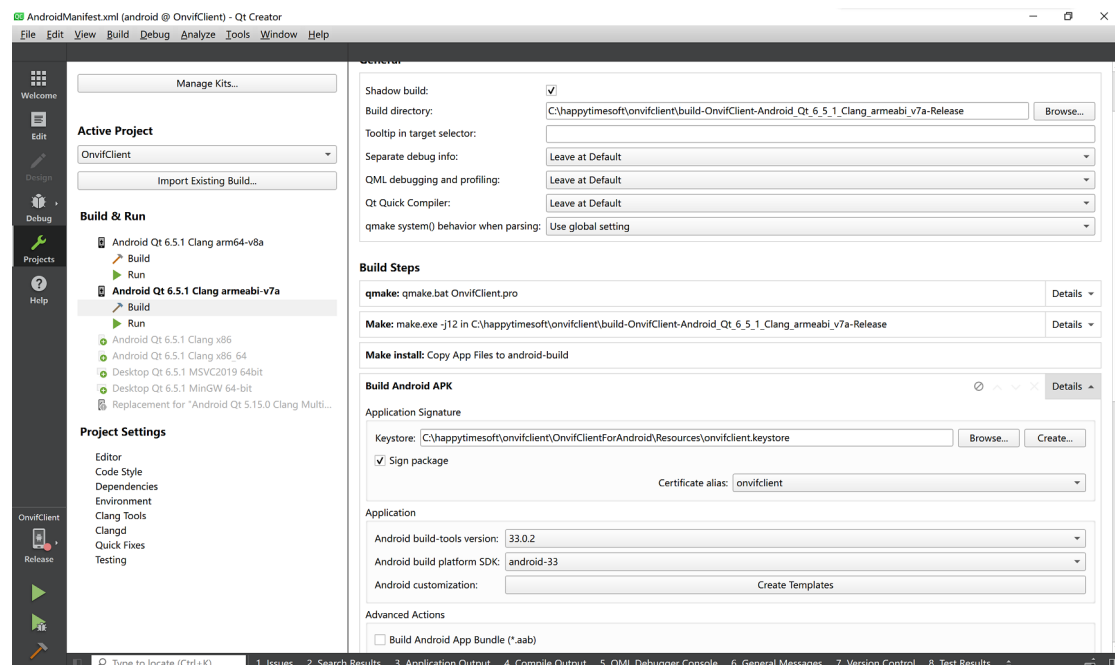
Application name

Application icon

Permissions required

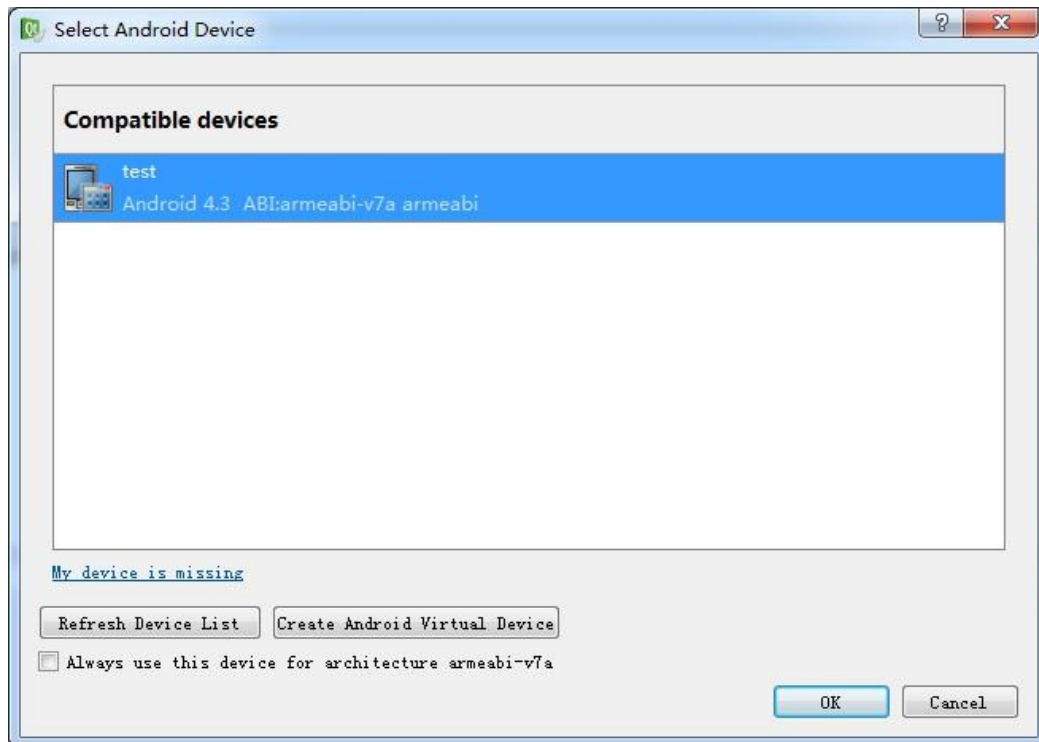
12.5 Build Release APK

Select projects, switch to run configurations page, as the following:



Click “Create...” button to create a sign packet, the check the “Sign packet”.

Click  button, pop up the following dialog:



Select the android device, then click “OK” button.

Note: you can connect your real device or create a virtual device